

Quiz 4 (B) :Introduction to Programming

Name: _____

Date: 12/12/2025

ERP: _____

Note: All questions from 1- 7 are 2 marks each.

1. Question

```
struct Node {  
    int data;  
    Node* next;};  
  
int main() {  
    Node* head = new Node{5, nullptr};  
    head->next = new Node{10, nullptr};  
    Node* ptr = head;  
    while (ptr != nullptr) {  
        cout << ptr->data << " ";  
        ptr = ptr->next;  
        if(ptr == head->next)  
            break;  
    } return 0; }
```

Options:

- A. 5
- B. 5 10
- C. Infinite loop
- D. 10
- E. None

2. Question

```
struct Node {  
    int data;  
    Node* next; };  
  
bool hasLoop(Node* head) {  
    if(head == nullptr) return false;  
    Node* slow = head;  
    Node* fast = head->next;  
    while(fast != nullptr && fast->next != nullptr) {  
        if(slow == fast) return true;  
        slow = slow->next;  
        fast = fast->next->next;    }  
    return false; }  
  
int main() {  
    Node* head = new Node{1, nullptr};  
    cout << hasLoop(head) << endl;  
    return 0; }
```

Options:

- A. true
- B. false
- C. Runtime error
- D. Compilation error
- E. None

3. Question

```
struct Node {  
    int data;  
    Node* next; };  
  
void modifyList(Node* head) {  
    if(!head || !head->next) return;  
    Node* temp = head->next;  
    head->next = temp->next;  
    delete temp; }  
  
int main() {  
    Node* head = new Node{7, nullptr};  
    head->next = new Node{14, nullptr};  
    head->next->next = new Node{21, nullptr};  
    modifyList(head);  
    cout << head->next->data << endl;  
    return 0; }
```

Options:

- A. 14
- B. 21
- C. Segmentation fault
- D. 7
- E. None

4. Question

```
struct Node {  
    int data;  
    Node* next;};  
  
int findElement(Node* head, int pos) {  
    if(!head) return -1;  
    while(pos > 0 && head) {  
        head = head->next;  
        pos--; }  
    return head ? head->data : -1;}  
  
int main() {  
    Node* head = new Node{3, nullptr};  
    head->next = new Node{6, nullptr};  
    cout << findElement(head, 2) << endl;  
    return 0;}
```

Options:

- A. 3
- B. 6
- C. -1
- D. Runtime error
- E. None

[Note: All tracing questions from 5-7 are 3 marks each]

5. Question:

```
struct Node {  
    int data;    Node* next;};  
  
void reorderList(Node* head) {  
    if(!head || !head->next) return;  
  
    Node* temp = head;  
    while(temp->next) {  
        Node* last = temp->next;  
        if(last->next == nullptr) {  
            last->next = temp->next;  
            temp->next = last;  
            break;    }  
        temp = temp->next;    }  
}  
  
int main() {  
    Node* head = new Node{1, nullptr};  
    head->next = new Node{2, nullptr};  
    head->next->next = new Node{3, nullptr};  
    head->next->next->next = new Node{4, nullptr};  
    reorderList(head);  
    Node* ptr = head;  
    while(ptr != nullptr) {  
        cout << ptr->data << " ";    ptr = ptr->next;    }    return 0; }
```

6. Question:

```
struct Node {  
    int data;  
    Node* next; };  
  
Node* alternateNodes(Node* head) {  
    if(!head || !head->next) return head;  
    Node* curr = head;  
    while(curr && curr->next) {  
        Node* temp = curr->next;  
        curr->next = temp->next;  
        temp->next = nullptr;  
        delete temp;  
        curr = curr->next;  
    } return head; }  
  
int main() {  
    Node* head = new Node{1, nullptr};  
    head->next = new Node{2, nullptr};  
    head->next->next = new Node{3, nullptr};  
    head->next->next->next = new Node{4, nullptr};  
    head->next->next->next->next = new Node{5, nullptr};  
    head = alternateNodes(head);  
    Node* ptr = head;    while(ptr != nullptr) {    cout << ptr->data << " ";    ptr = ptr->next;    }    return 0; }
```

7. Question:

```
struct Node {  
    int data;  
    Node* next; };  
  
int getIntersection(Node* list1, Node* list2) {  
    Node *ptr1 = list1, *ptr2 = list2;  
    while(ptr1 != ptr2) {  
        ptr1 = ptr1 ? ptr1->next : list2;  
        ptr2 = ptr2 ? ptr2->next : list1;  
    }    return ptr1 ? ptr1->data : -1; }  
  
int main() {  
    Node* list1 = new Node{1, nullptr};  
    list1->next = new Node{2, nullptr};  
    list1->next->next = new Node{3, nullptr};  
    Node* list2 = new Node{4, nullptr};  
    list2->next = list1->next; // list2 intersects with list1 at node with data=2  
    cout << getIntersection(list1, list2) << endl;  
    return 0; }
```

Q8. [5 marks]

The following code attempts to remove duplicate consecutive nodes from a linked list.

Identify and fix the bugs in the implementation:

```
struct Node {  
    int data;  
    Node* next; };  
  
Node* removeConsecutiveDuplicates(Node* head) {  
    Node* current = head;  
    while(current && current->next) {  
        if(current->data == current->next->data) {  
            current->next = current->next->next;    }  
        current = current->next;    }  
    return head; }
```